Can a marketplace spot late deliveries on the day they are ordered?

Purchase-time delay triage under a daily review budget · Olist, 2016–2018

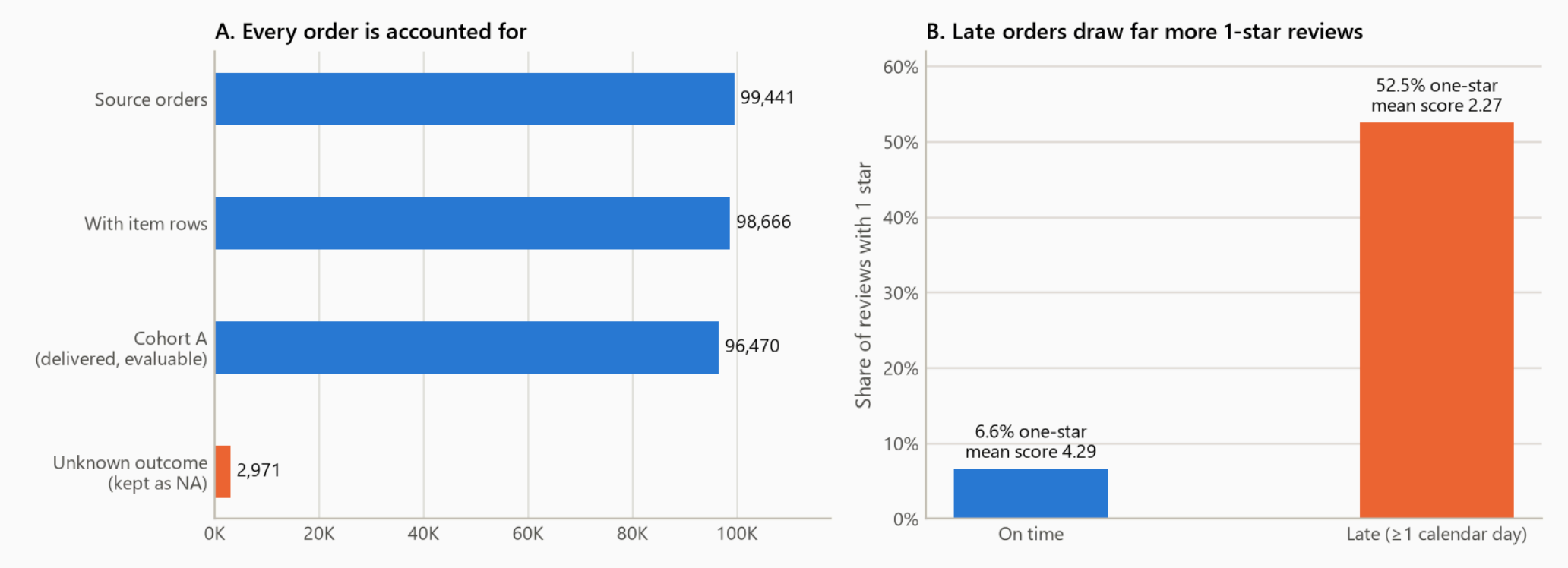
IS5126 Final Project · Group _[number and members]_

Short answer: only weakly. Simple route rules beat 24 machine-learning models in two separate test periods, and a later signal (the seller missing the shipping deadline) is more precise in both.

Late deliveries drive the worst reviews

Why this decision matters

- 6.8% of 96,470 delivered orders arrived after the promised date
- 52.5% of late orders got a one-star review, against 6.6% of on-time orders
- Mean review score: 2.27 when late, 4.29 when on time
- The promise date is shown at checkout, so the marketplace takes the blame



The decision we model

An evening review queue with a fixed budget

- Each evening, the operations team reviews at most `floor(0.10 × N)` of the N orders placed that day (about 20 a day)
- Only information known at purchase may be used
- No carry-over of unused capacity; the workload stays stable even when the late rate swings

> Q1: how many late orders can the queue catch, at what false-alarm cost?

> Q2: do machine-learning models beat simple rules?

> Q3: is acting at purchase time worth it, or is a later checkpoint better?

Data: every order accounted for

Public Olist dataset · 9 tables · purchases 2016-09-04 to 2018-10-17

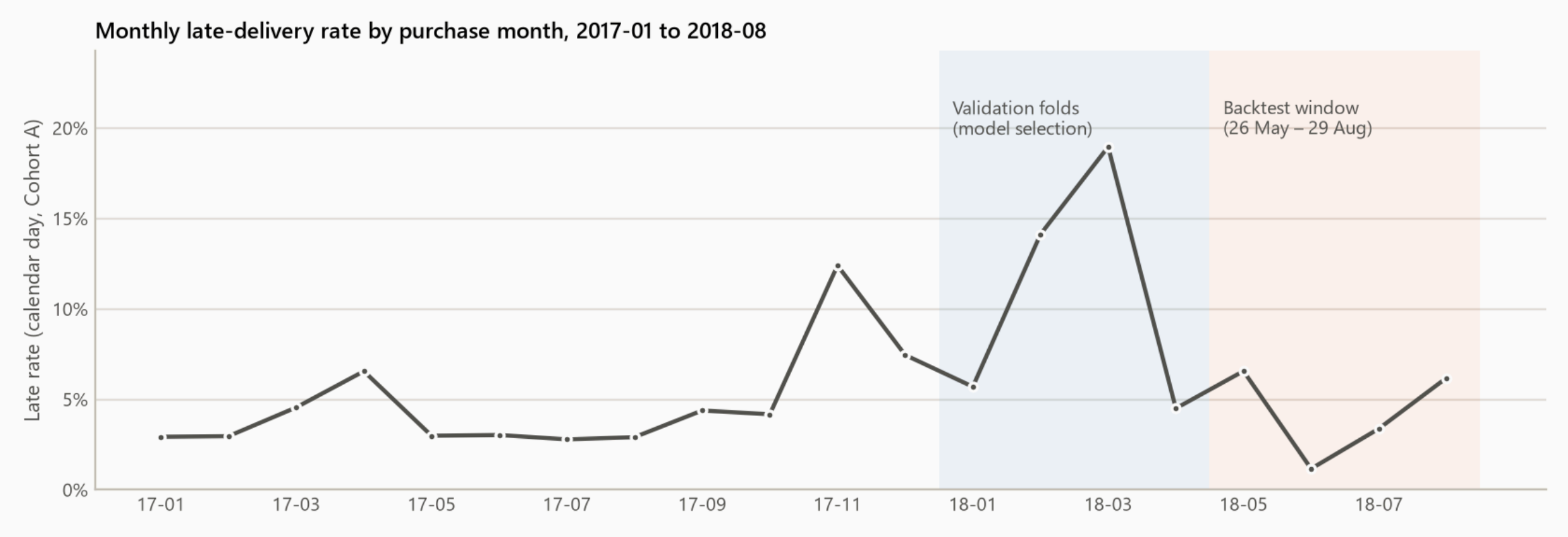
Group	Orders	Use
All source orders	99,441	Scored in the operational view
Delivered with valid dates (Cohort A)	96,470	Labelled; training and metrics
Unknown outcome (shipped, canceled, …)	2,971	Kept as missing, never "on time"

- **Late** = delivered on a calendar day after the promised date: 6,534 orders (6.77%)
- Strict clock-time label as a check: 7,826 (8.11%); the difference is 1,292 same-day deliveries

The late rate is not stable

Exploration: drift and right-censoring

- 5.6% in 2017; 12.4% in November 2017; 19.0% in March 2018; 1.2% in June 2018
- A model trained in one period is always tested in a different one
- At the selection date, 104 April orders were still undelivered, and 89 of them ended late: April looked like 3.24% but was 4.50%



Method: evaluate in time order

29 candidates · chosen on Jan–Apr 2018 · tested once on 26 May–29 Aug 2018

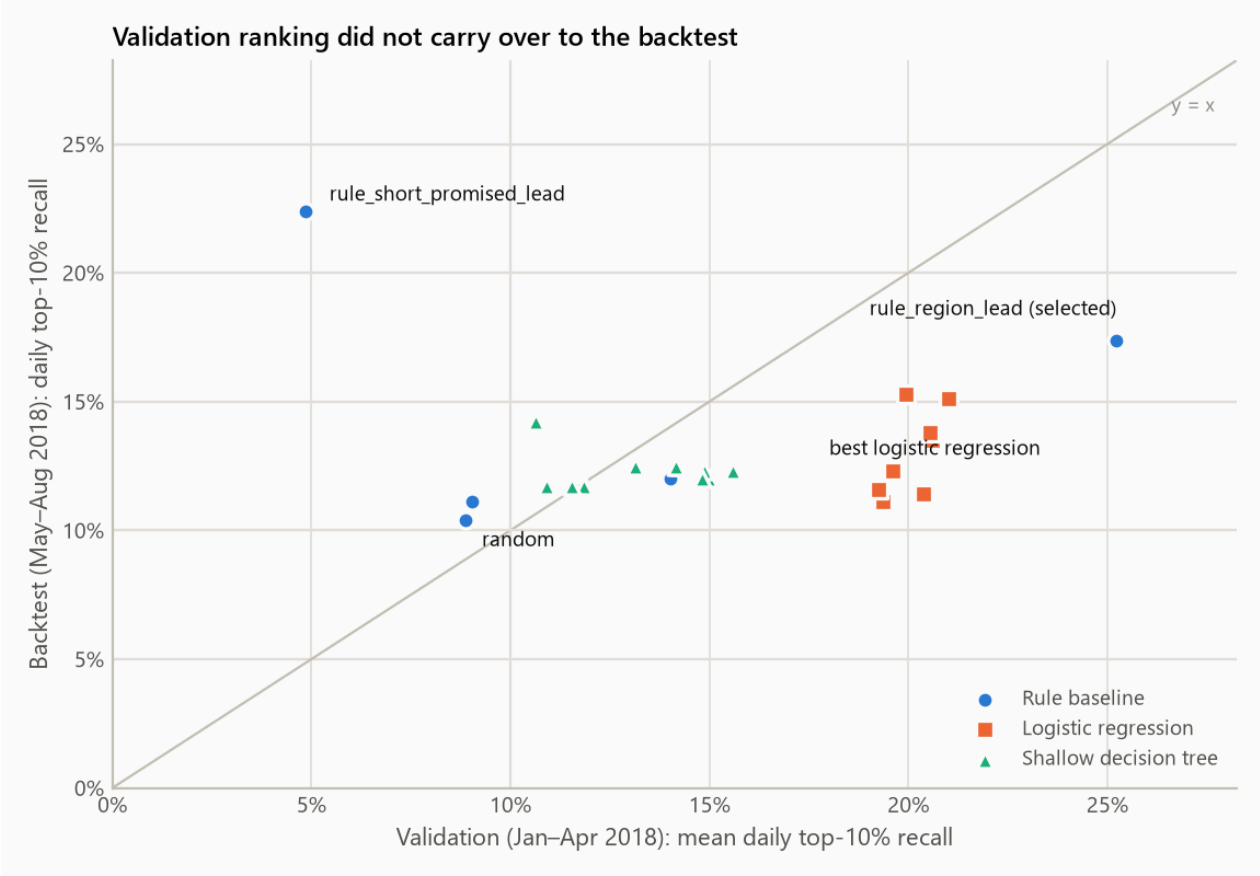
- **5 rules:** random, constant prior, shortest promise first, seller history, route history × promise tightness
- **24 models:** 8 logistic regressions, 16 shallow trees; 74 purchase-time features (+7 payment features tested separately)
- Training uses only orders already delivered at each cutoff
- **Selection rule fixed in advance:** highest mean daily top-10% recall over the 4 folds; a model must beat the best rule
- **Uncertainty:** weekly block bootstrap, 15 weeks, 2,000 draws

Result 1: a two-input rule beats every model

Route late rate × promise tightness (median lead 24 days ÷ this order's lead)

Candidate	Validation recall	Worst fold	Backtest recall
Route-promise rule (selected)	**25.2%**	23.2%	**17.4%**
Best logistic regression	21.0%	2.9%	15.1%
Best shallow tree	15.6%	12.6%	12.3%
Random order	8.9%	7.2%	10.4%

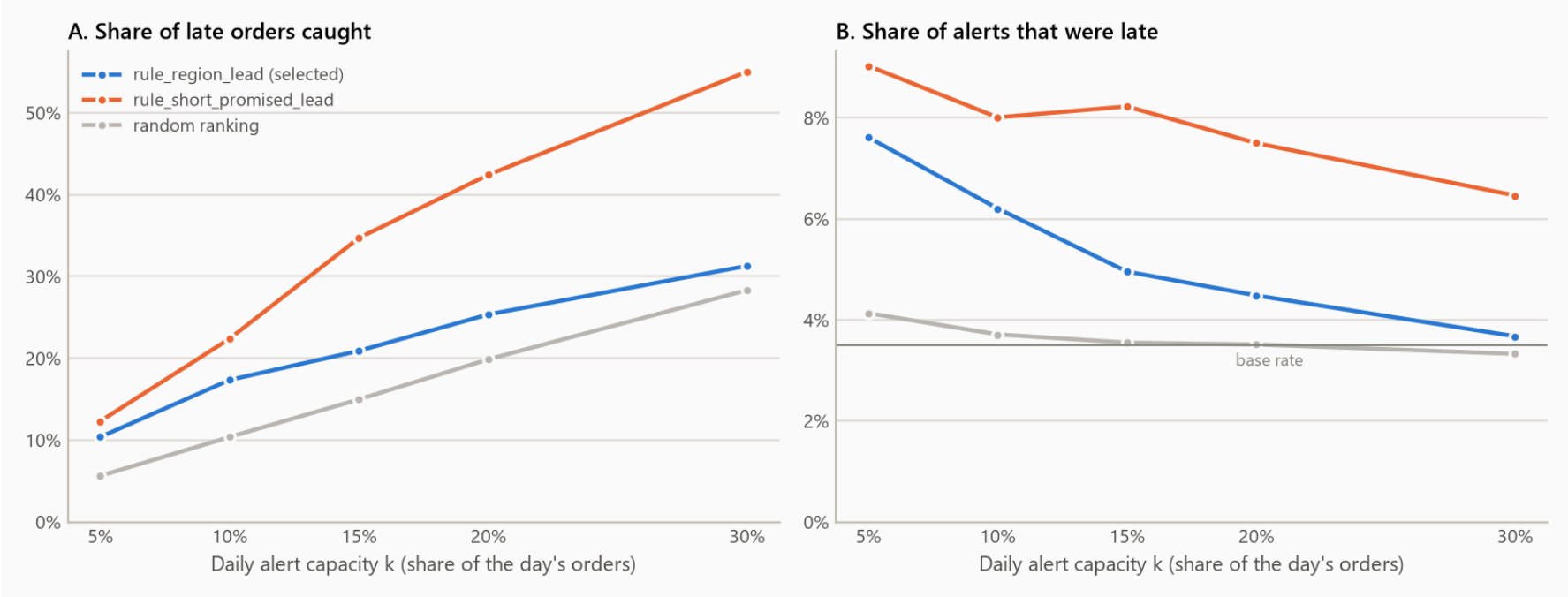
- Payment features added nothing material
- No model beat the rule in the backtest either



Result 2: the purchase-time signal is weak

Backtest, daily top 10%, 674 late orders among 19,294

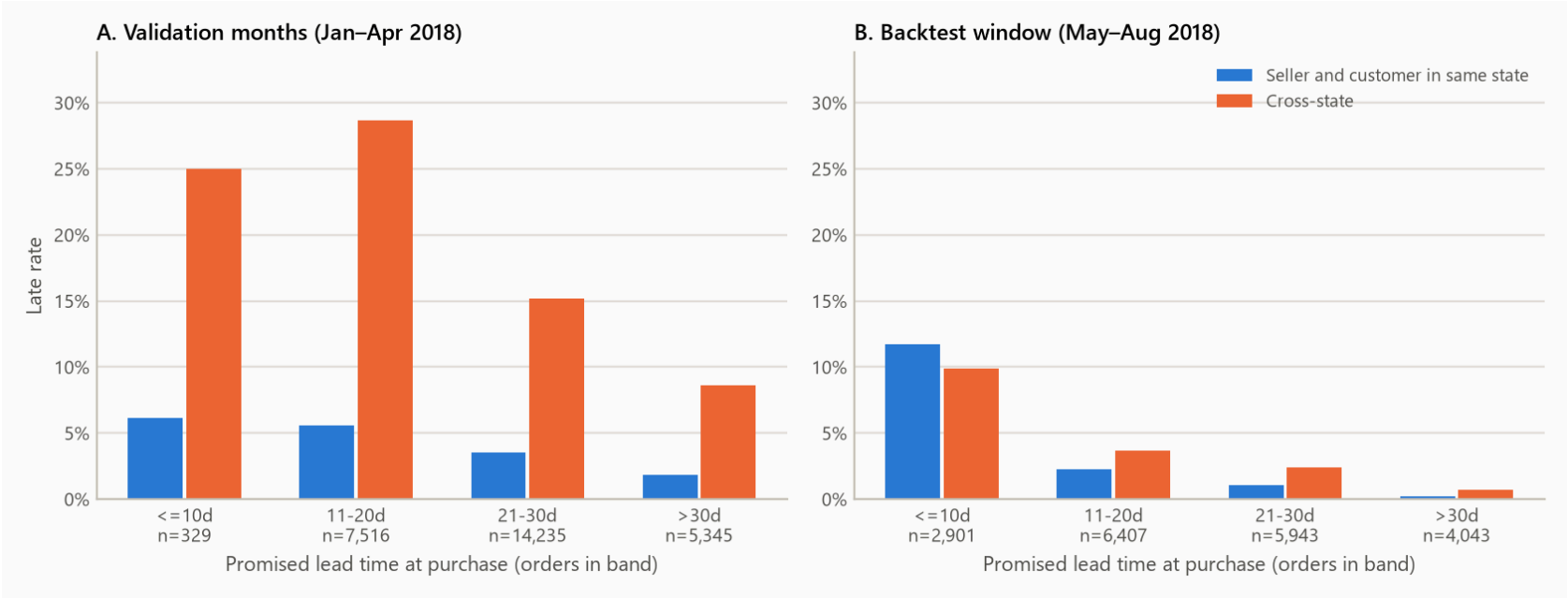
- 1,885 alerts → 117 late orders caught: **17.4% recall, 6.2% precision** (base rate 3.49%, lift 1.78)
- 16.1 orders reviewed per late order found
- 95% intervals (weekly blocks, 2,000 draws): recall 12.7% to 27.0%, precision 4.2% to 8.4%
- Unknown-outcome orders take 45 of 1,922 slots, so precision lies between 6.0% and 8.4%
- Precision falls as capacity grows: 7.6% at 5%, 3.7% at 30% (lift 1.05)



Result 3: what makes an order late changed

Validation months vs backtest window

- **Jan–Apr 2018:** cross-state orders 14.2% late vs 4.9% within a state; 83.7% of late orders crossed a state line
- **May–Aug 2018:** within-state 5.1% vs cross-state 2.4%; promises of 10 days or less rose from 1.2% to 15.0% of orders and were late 11.6% of the time
- São Paulo → São Paulo short promises alone: 41.4% of all late orders
- The shortest-promise rule went from last to first, but its edge is not significant (−13.4 pp to +13.9 pp), and we did not switch after seeing the test

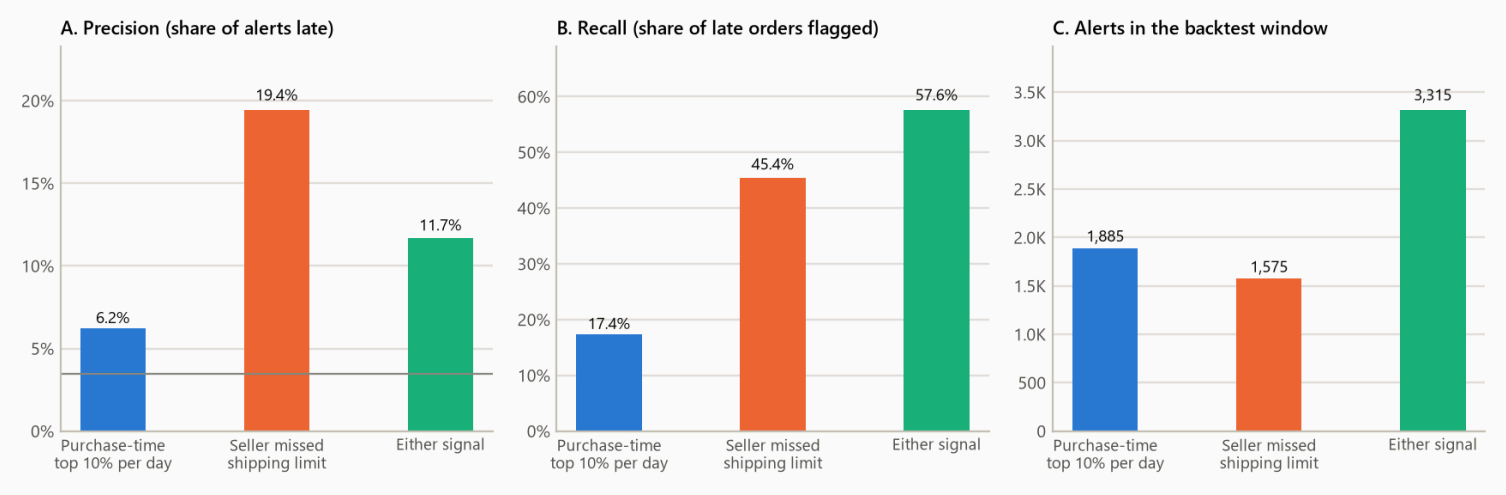


Result 4: a later signal is far sharper (post hoc)

Flag an order if the seller has not handed it to the carrier by the shipping-limit date

Policy	Alerts	Precision	Recall
Purchase-time top 10%	1,885	6.2%	17.4%
Seller missed shipping limit	**1,575**	**19.4%**	**45.4%**
Either signal	3,315	11.7%	57.6%

- About 16 alerts a day; precision gain of 13.2 points (interval 4.3 to 20.1 points)
- Late orders still had a median of 3.2 days before the promise when flagged
- Designed after seeing the backtest: a hypothesis to pilot

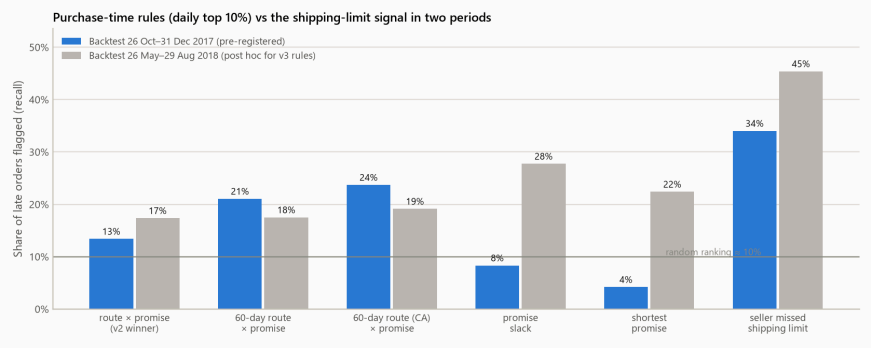


Result 5: a second, pre-registered test (late 2017)

Design frozen before any model saw this period · select Jun–Sep 2017 · test 26 Oct–31 Dec 2017

Candidate	Validation recall	Backtest recall	Backtest precision
60-day route × promise (selected)	**19.1%**	**21.1%**	**21.6%**
Route × promise (v2 winner)	18.1%	13.4%	13.7%
Best model (shallow tree)	18.7%	17.3%	17.7%
Promise slack	16.0%	8.3%	8.5%
Seller missed shipping limit	—	34.0%	28.9%

- Models again did not beat the rules; recent route history gained 7.7 points of recall (interval 4.5 pp to 12.8 pp)
- Rules that looked best in hindsight on 2018 failed here (promise slack 8.3%, shortest promise 4.2%)
- The shipping-limit signal held on unseen data: +7.3 points of precision (interval 1.4 to 13.6 points)



Is acting worth it?

Three cost scenarios fixed in the configuration before any run

Scenario	Break-even precision	Purchase-time queue	Shipping-limit signal
Conservative	22.7%	Loses at every budget	−5.9
Moderate	7.8%	Loses (−7.8 per 1,000 at 10%)	**+49.0**
High SLA penalty	3.0%	**+31.9** per 1,000 at 10%	**+135.7**

- 2018 window shown; at 6.2% precision a purchase-time queue needs a caught delay to be worth about 16 times a contact
- In the high-delay late-2017 window (21.6% precision) the same queue earns +69.4 per 1,000 orders in the moderate scenario: value depends on the current late rate
- Values are hypothetical (configuration money units); only a pilot can show what an intervention prevents

Recommendations

- **No large human review queue at purchase time;** size any purchase-time effort to the current late rate
- **If ranking at purchase time, use recent (60-day) route history** and only the top of each day's list
- **Move human follow-up to the shipping-limit checkpoint:** more precise than the purchase-time queue in both periods (19.4% vs 6.2%; 28.9% vs 21.6%); pilot it first
- **Review promise-setting for short local routes:** São Paulo → São Paulo promises of 10 days or less produced 41% of 2018 late orders
- **Monitor monthly and re-select** on recent folds

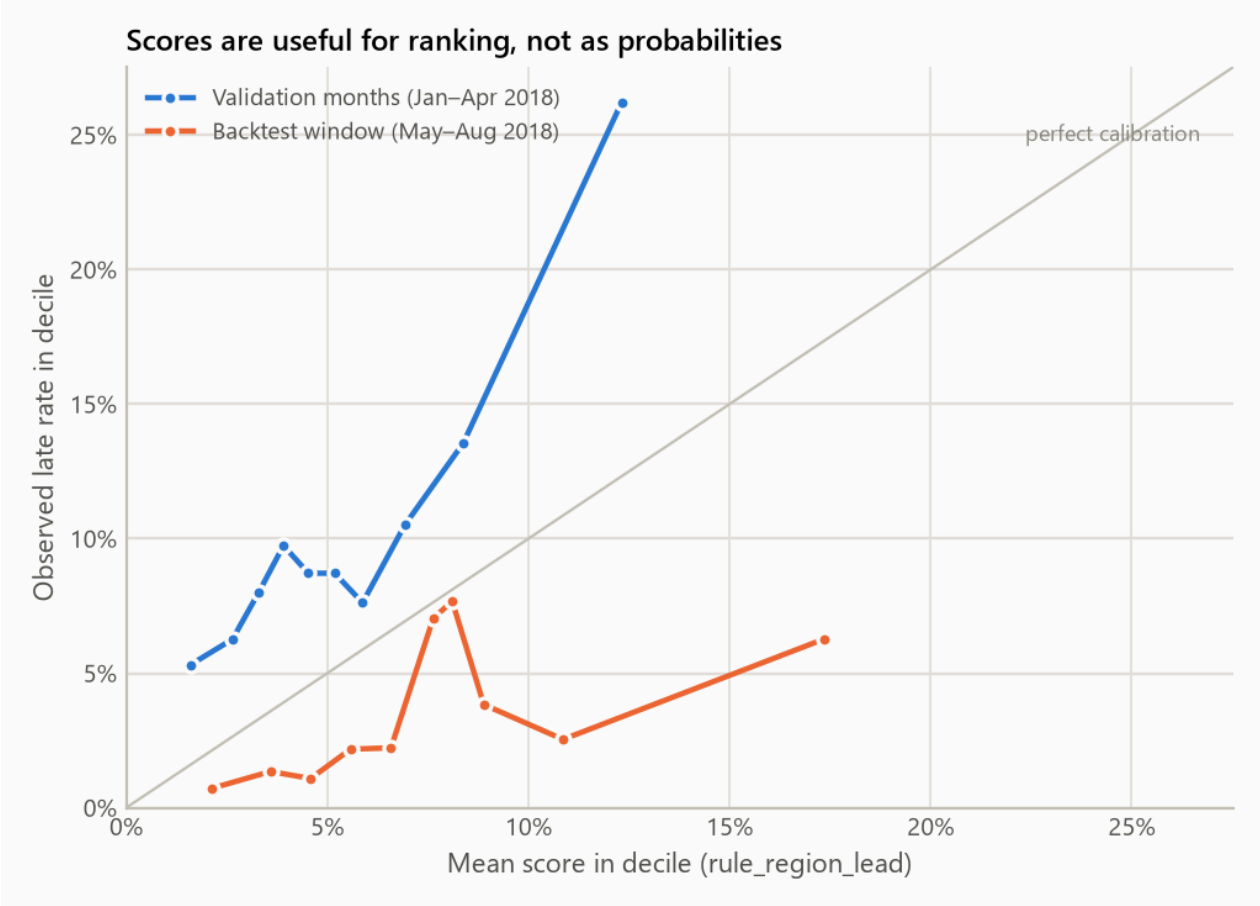
Limitations and conclusion

- **Data:** one marketplace in 2016–2018; the label depends on Olist's own promise algorithm; timestamps may lag; 2,971 outcomes unknown
- **Method:** short test windows (96 and 67 days) with wide intervals; the 2018 window is a diagnostic holdout; the second test is unseen but earlier in time than 2018, and its ideas came from studying 2018
- **Conclusions:** associations only; cost values are illustrative; two analyses are post hoc

> At purchase time, delays are only weakly predictable and the pattern shifts within months. Recent route history is the robust improvement; the bigger value lies in a later checkpoint and in fixing short local promises.

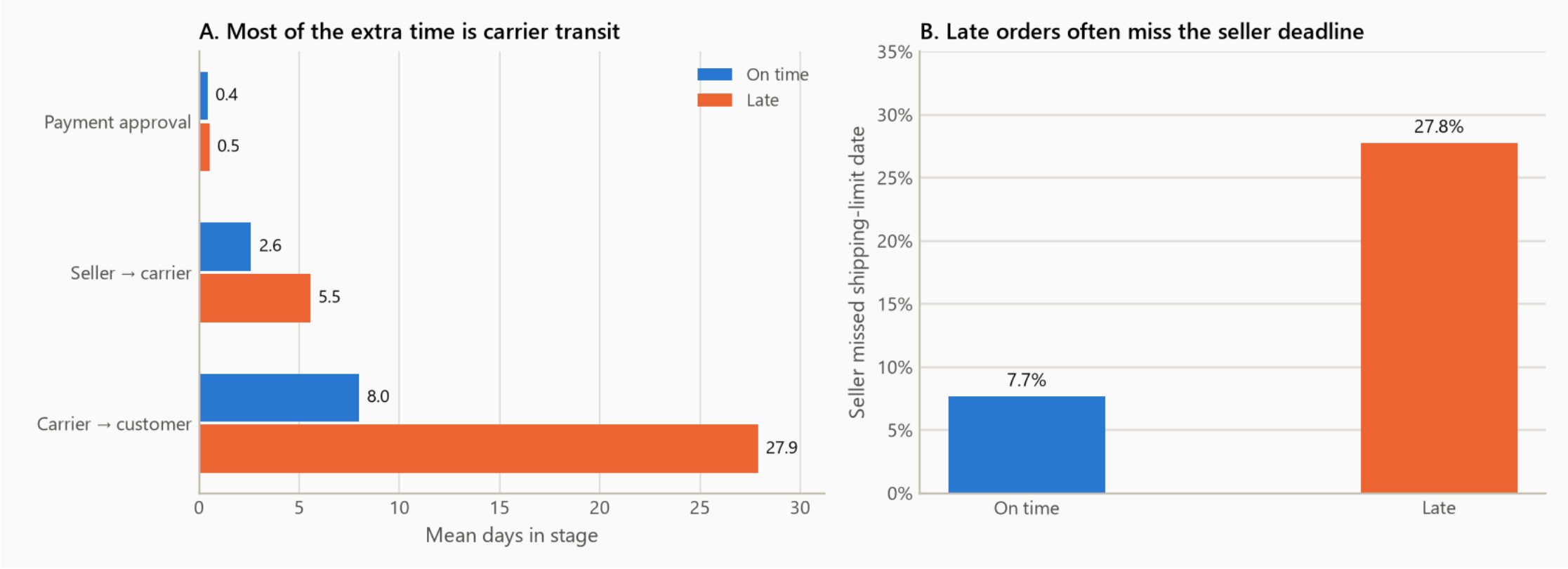
Backup: scores are rankings, not probabilities

- Validation months: mean score 5.5% vs 10.5% observed (under-prediction)
- Backtest: mean score 7.5% vs 3.5% observed (ratio 2.16)
- Top backtest decile: 17.3% predicted, 6.3% observed
- This is why the queue uses a daily quota, not a probability threshold



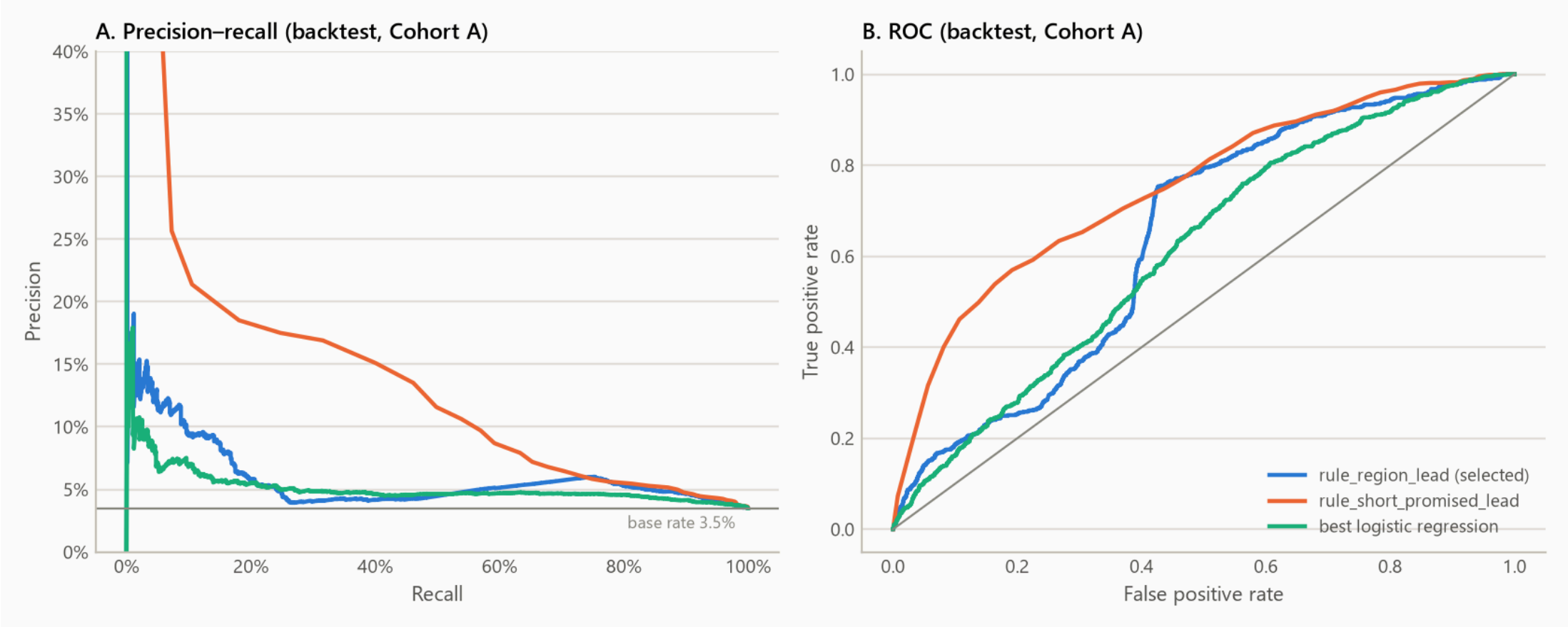
Backup: where the extra time goes

- Late orders took 22.9 more days than on-time orders
- Carrier transit accounts for 19.9 of those days (87%)
- Seller hand-off accounts for 2.9 days (13%)
- 27.8% of late orders missed the seller's shipping limit, against 7.7% of on-time orders



Backup: ranking curves in the backtest

- Selected rule: average precision 0.0604 (95% interval 0.0464 to 0.0805), ROC-AUC 0.6416
- Shortest-promise rule: average precision 0.1203, ROC-AUC 0.7503
- Best logistic regression: average precision 0.0516, ROC-AUC 0.615



Backup: reproducibility

- Formal run `experiment-v2-001` is frozen with its code and configuration hashes; raw data are checked against SHA-256 digests
- `scripts/report_analysis.py` rebuilds every table and figure in the report from the frozen predictions, and stops if the formal run's headline counts do not reproduce exactly
- `scripts/check_docs_consistency.py` rejects any number in the report, slides or briefs that is not traceable to an artifact, and any broken figure link
- One command runs all acceptance gates: `scripts/acceptance.ps1`